

AI ASSISTED CODING

NAME: G.Vijaya Simha Reddy

ROLLNO:2503A52671

BATCH:23

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task 1: Privacy in API Usage

Prompt: Generate Python code to fetch weather data securely without exposing API keys.

A. Insecure Version (Hardcoded API Key)

```
import requests
```

```
API_KEY = "123456789abcdef" # Hardcoded (Unsafe)
```

```
CITY = "Warangal"
```

```
url = f"https://api.openweathermap.org/data/2.5/weather?q={CITY}&appid={API_KEY}&units=metric"
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
print("Temperature:", data["main"]["temp"], "°C")
```

Privacy Risk

- API key is visible in the code.
- Anyone can misuse the key.

- Unsafe when uploaded to GitHub.

B. Secure Version (Environment Variable)

```
import os
```

```
import requests
```

```
API_KEY = os.getenv("WEATHER_API_KEY")
```

```
CITY = "Warangal"
```

```
url = f"https://api.openweathermap.org/data/2.5/weather?q={CITY}&appid={API_KEY}&units=metric"
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
print("Temperature:", data["main"]["temp"], "°C")
```

Task 2: Privacy & Security in File Handling

A. Insecure Version

```
name = input("Enter Name: ")
```

```
email = input("Enter Email: ")
```

```
password = input("Enter Password: ")
```

```
with open("users.txt", "a") as file:
```

```
    file.write(f"{name},{email},{password}\n")
```

```
print("Data Saved.")
```

Privacy Risks

- Password stored as plain text.
- Anyone opening the file can read passwords.
- High security risk.

B. Secure Version (Password Hashing)

```
import hashlib

name = input("Enter Name: ")
email = input("Enter Email: ")
password = input("Enter Password: ")

hashed_password = hashlib.sha256(password.encode()).hexdigest()

with open("users.txt", "a") as file:
    file.write(f"{name},{email},{hashed_password}\n")

print("Data Saved Securely.")
```

Why Hashing is Better

- Original password cannot be directly recovered.
- Protects user credentials.
- Common industry practice.

Task 3: Transparency in Algorithm Design

Armstrong Number Program

```
number = int(input("Enter a number: "))

digits = len(str(number))
temp = number
total = 0

while temp > 0:
    digit = temp % 10
    total += digit ** digits
    temp //= 10

if total == number:
    print(number, "is an Armstrong Number.")
else:
    print(number, "is not an Armstrong Number.")
```

Input:

153

Output:

153 is an Armstrong Number.

Task 4: Transparency in Algorithm Comparison**Bubble Sort**

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]

    return arr

numbers = [5,3,8,1,2]
print("Bubble Sort:", bubble_sort(numbers))
```

How Bubble Sort Works

- Compare adjacent numbers.
- Swap if larger.
- Repeat until sorted.

Task 5: Transparency in AI Recommendations**Product Recommendation System**

```
products = {
    "Student": [
        ("HP OmniBook 3", "Good for coding, VS Code, and 4-year college use."),
        ("Lenovo IdeaPad", "Affordable and suitable for assignments.")
    ],
    "Gamer": [
        ("ASUS TUF", "Powerful graphics for gaming."),
        ("Acer Nitro", "Good performance for games.")
    ]
}
```

```
user = input("Enter category (Student/Gamer): ")
```

```
if user in products:
```

```
    print("\nRecommended Products:\n")
```

```
    for product, reason in products[user]:
```

```
        print("Product:", product)
```

```
        print("Reason :", reason)
```

```
    print()
```

```
else:
```

```
    print("Category not found.")
```

Input:

Student

Output:

Recommended Products:

Product: HP OmniBook 3

Reason : Good for coding, VS Code, and 4-year college use.

Product: Lenovo IdeaPad

Reason : Affordable and suitable for assignments.

Evaluation of Transparency

- Recommendations include clear reasons.
- Users understand why each product is suggested.
- Improves trust and explainability.